오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 심화

DAY 21

모듈 시스템, NPM, Vite

export · import로 파일을 모듈로 나누고,
npm · 번들러 · Vite의 개념을 익힙니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **export · import로 파일을 모듈로 나눌 수 있다**

   이름으로 · 기본으로 내보내고 가져오며, 모듈 script가 일반 script와 무엇이 다른지 압니다.

2. **npm과 package.json이 무엇인지 설명할 수 있다**

   패키지를 관리하는 npm과, dependencies · devDependencies, 버전 기호 ^ · ~의 뜻을 압니다.

3. **빌드와 번들러, Vite가 무엇인지 설명할 수 있다**

   여러 파일을 묶는 번들러와 빌드 도구 Vite를 압니다 — 직접 써 보는 것은 DAY 27.

오늘의 결과물

모듈 셋으로 나눈 댓글 보기(실습 01)와, 저장 · 날짜 · 연결로 나눈 할 일 목록(실습 02)

ROADMAP

오늘의 흐름

01

모듈

export · import
모듈 script · 실습 01

실습 포함

02

NPM

패키지와 npm
package.json · 버전

03

Vite

빌드 · 번들러
Vite

04

모듈 분리

폴더 구조 설계
나누는 기준 · 실습 02

실습 포함

첫 섹션은 브라우저만으로 파일을 모듈로 나눠 보고, 둘째 · 셋째 섹션에서 npm · 번들러 · Vite의 개념을 봅니다.
마지막 섹션은 DAY 20의 할 일 목록을 역할마다 모듈로 나눕니다 — 두 실습 모두 Live Server로 엽니다.

되짚기 · 01

DAY 20에 역할별로 나눈 코드

배운 곳	그때 본 것	오늘
DAY 20 16장	통신 · 화면 · 흐름 — 함수 셋으로	역할마다 파일 하나 — 모듈
DAY 20 12장	lib.js를 script로 먼저 불러 함께 씀	export · import로 주고받기
DAY 20 실습 01	댓글 보기 — comments.js 한 파일	실습 01 — 모듈 셋으로
DAY 20 실습 02	할 일 목록 — todo.js 한 파일	실습 02 — 저장 · 날짜 · 연결로

오늘은 함수가 아니라 파일을 나눕니다

파일 하나에 한 가지 일을 두고, 필요한 것만 가져다 씁니다.

script 태그 여러 개 — 순서가 약속

```
<!-- order/index.html — 순서가 바뀜 -->
<script src="main.js"></script>
<script src="lib.js"></script>

// lib.js
function greet(name) {
  return `${name}님, 반가워요`;
}

// main.js
console.log(greet("김오즈"));
```

일반 script의 약속

위에서부터 차례로 실행 — 먼저 불러야 쓸 수 있음
맨 바깥 이름은 모두 전역 — 이름이 부딪칠 수 있음

모듈에서는 — 오늘 배울 것

import로 직접 가져오고, 이름은 파일마다 따로

```
▶ Uncaught ReferenceError: greet is not defined                main.js:1
    at main.js:1:9

>
```

Console (위쪽 도구 막대는 잘라 냄)

준 비

오늘의 셋팅

```
Day21/
  practice01/            ← 실습 01 (Live Server로 엶)
    index.html  style.css
    day20_comments.js    ← DAY 20 완성본 — 나눌 원본
    api.js  view.js  main.js
  practice02/            ← 실습 02 (Live Server로 엶)
    index.html  style.css
    day20_todo.js        ← DAY 20 완성본 — 나눌 원본
    storage.js  date.js  main.js
```

Node.js 설치는 DAY 27

오늘은 설치하지 않고,
npm · Vite는 개념만
알아 둡니다.

main에 바로 커밋

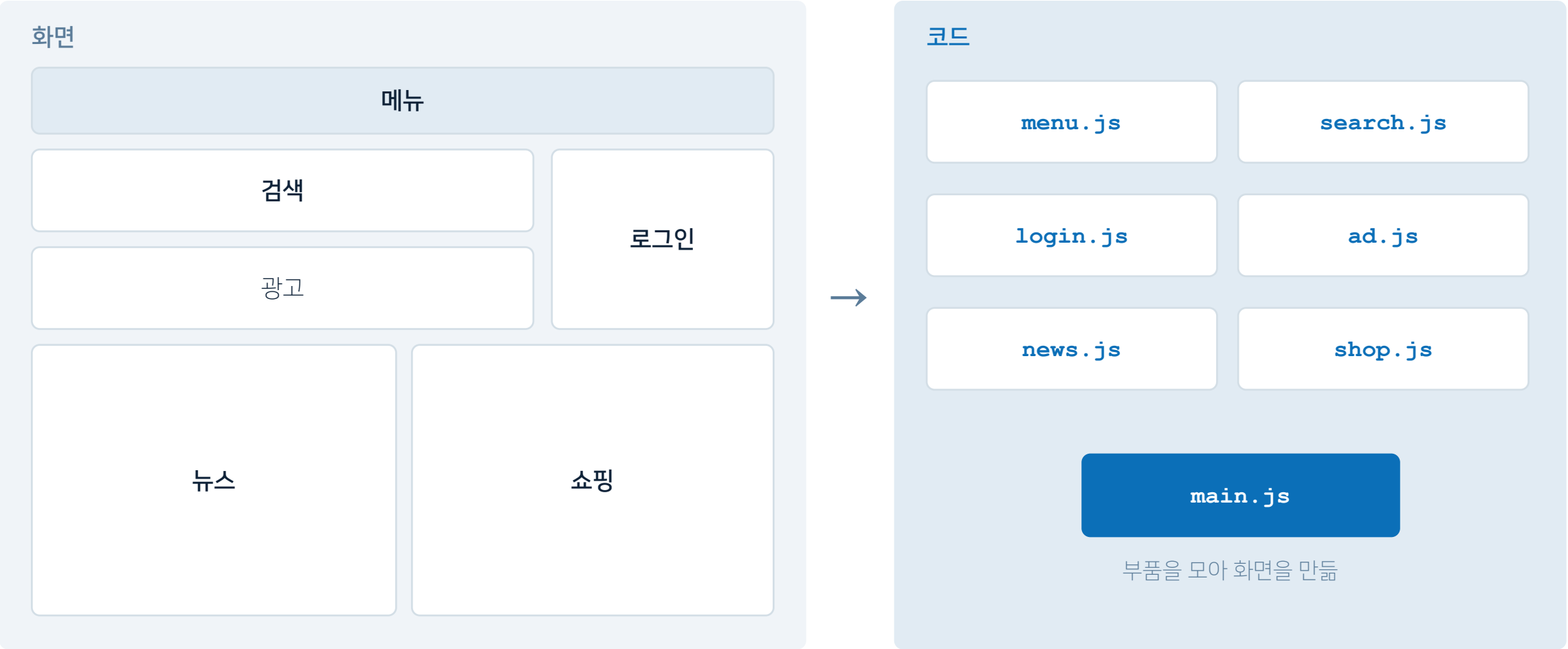
$ git status
브랜치는 DAY 22부터

01

모듈 — export와 import

부품으로 나누기 · export · import · type="module" · as · * as · default · 모듈의 특징

큰 화면도 부품으로 나눠 만들기



모듈 — 따로 떼어 낸 코드 파일 하나

SPLIT

나눠서

파일 하나에 한 가지 일.
고칠 곳을 찾기 쉽습니다.

EXPORT · IMPORT

골라서 주고받기

밖에 줄 것만 export,
쓸 것만 import 합니다.

REUSE

다시 쓰기

한 번 만든 모듈을
다른 파일에서도 가져다 씀.

export는 부품 상자에 넣기, import는 꺼내 조립하기

오늘은 export와 import, 이 두 말을 직접 써 봅니다.

내보내기(export)와 가져오기(import)

```
// printer.js
export const title = "CodePrinter";
export function print(value) {
  console.log(value);
}
// main.js
import { title, print } from "./printer.js";

print(title);
<!-- index.html -->
<script type="module" src="main.js"></script>
```

import { 이름 } from "./파일.js"

export — 이름 앞에 붙여 내보냄
중괄호 안에 가져올 이름
from 뒤에 그 파일의 경로

```
  CodePrinter                                    printer.js:3
>
```

Console (위쪽 도구 막대는 잘라 냄)

01 · type="module"

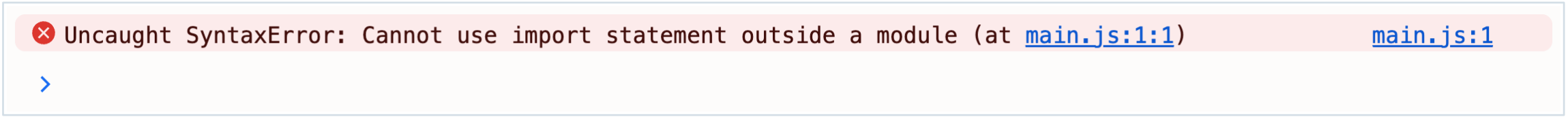
import가 있는 파일은 모듈로 불러오기

```
<!-- 일반 script로 불러오면 -->
<script src="main.js"></script>

<!-- 모듈로 불러오면 — 시작점 하나만 -->
<script type="module" src="main.js"></script>
```

index.html에는 시작점 하나

main.js가 printer.js를 import —
나머지는 브라우저가 따라가 받음



type 없이 불러온 Console (위쪽 도구 막대는 잘라 냄)

직접 열면 동작하지 않는 모듈

```
// file:///…/basic/index.html 로 열었을 때 — Chrome 154 Console 문구 (주소는 줄임)
Access to script at 'file:///…/basic/main.js' from origin 'null' has been blocked by CORS policy:
Cross origin requests are only supported for protocol schemes: chrome, …, data, http, https, isolated-app.
Failed to load resource: net::ERR_FAILED
```

FILE://

직접 연 파일

모듈을 받지 못해 화면이 비고
Console에 CORS 오류

LIVE SERVER

http://127.0.0.1:5500

서버로 열면 모듈이 동작.
VS Code 오른쪽 아래 Go Live

./ 로 시작해 .js까지 적는 경로

```
// main.js — ./ 를 빠뜨림
import { title, print } from "printer.js";

print(title);
```



Console (위쪽 도구 막대는 잘라 냄)

적은 경로	결과
`"./printer.js"`	같은 폴더의 printer.js — 됨
`"printer.js"`	TypeError — ./ 가 없음
`"./printer"`	404 — 확장자를 붙여 주지 않음

가져오면서 이름 바꾸기 — as

```
// printer.js — 10장과 같음
export const title = "CodePrinter";
export function print(value) {
  console.log(value);
}

// main.js
import { title as printTitle, print as pt } from "./printer.js";

pt(printTitle);
```

원래 이름 as 새 이름

title → printTitle

print → pt

이 파일 안에서는 새 이름으로

```
  CodePrinter                                    printer.js:3
>
```

Console (위쪽 도구 막대는 잘라 냄)

내보낸 것을 한꺼번에 — * as

```
// main.js — printer.js는 10장과 같음
import * as printer from "./printer.js";

printer.print(printer.title);
console.log(printer);
```

printer — 모듈 하나가 객체로

printer.title · printer.print
내보낸 것이 모두 속성으로
이름이 많을 때 한꺼번에



Console (위쪽 도구 막대는 잘라 냄)

01 · default

파일의 대표 하나 — export default

```
// printer.js
const title = "CodePrinter";
function print(value) {
  console.log(value);
}

export default { title, print };
```

```
// main.js
import printer from "./printer.js";
import myPrinter from "./printer.js";

printer.print(printer.title);
console.log(printer === myPrinter);
```

```
CodePrinter                                  printer.js:3
true                                         main.js:5
>
```

Console (위쪽 도구 막대는 잘라 냄)

중괄호 없이, 이름은 마음대로

한 파일에 하나만
printer · myPrinter 모두 같은 것
— true

이름으로 내보내기와 기본으로 내보내기

	이름으로 (named)	기본으로 (default)
내보내기	`export function print() {}`	`export default printer;`
한 파일에	여럿	하나만
가져오기	`import { print } from "./p.js";`	`import printer from "./p.js";`
가져올 이름	내보낸 이름 그대로 (바꾸려면 as)	가져오는 쪽이 정함

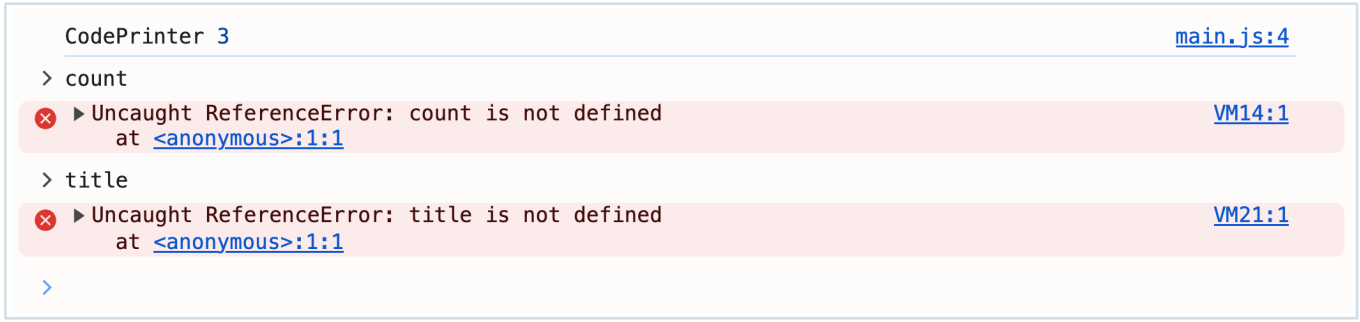
```
// 함께 가져오기 — 기본 이름 먼저, 이름으로 내보낸 것은 중괄호로
import greet, { shopName, price } from "./shop.js";
```

그 파일 안에서만 보이는 모듈 안의 이름

```
// main.js — printer.js는 10장과 같음
import { title } from "./printer.js";

const count = 3;
console.log(title, count);

// 그다음 Console 입력란에
> count
> title
```



Console (위쪽 도구 막대는 잘라 냄)

전역에 놓이지 않음

Console에서도 보이지 않음
쓰려면 export · import
값은 파일 안에서 console.log

모듈은 문서를 다 읽은 뒤 실행

```
// head 안의 script
const titleEl = document.querySelector("h1");
console.log(titleEl.textContent);
```

일반 script — null

```
▶ Uncaught TypeError: Cannot read properties of null (reading 'textContent')
    at main.js:2:21                                                    main.js:2
>
```

모듈 — 「문서를 다 읽은 뒤에」

```
문서를 다 읽은 뒤에                                                    main.js:2
>
```

모듈은 자동 defer — MDN에 적힌 모듈의 특징

여러 곳에서 가져와도 한 번만 실행되는 모듈

```
// shared.js
console.log("shared.js 실행");

export const appName = "DAY 21";

// a.js
import { appName } from "./shared.js";

console.log("a.js :", appName);

// b.js
import { appName } from "./shared.js";

console.log("b.js :", appName);

// main.js
import "./a.js";
import "./b.js";
```

```
shared.js 실행                    shared.js:1
a.js : DAY 21                     a.js:3
b.js : DAY 21                     b.js:3
›
```

Console (위쪽 도구 막대는 잘라 냄)

shared.js 실행 — 한 번

a.js · b.js가 같은 값을 받음
import "./a.js" — 실행만
함께 쓰는 값은 한 파일에

실습 01

댓글 보기를 모듈 셋으로 나누기

DAY 20 실습 01의 comments.js를 통신 · 화면 · 연결 세 파일로 나눕니다.
화면과 동작은 그대로이고, 파일끼리 export · import로 주고받습니다.

나누는 순서

① api.js — getJSON 옮기기, export 없이

② api.js — getComments(postId)를 export

③ api.js — toMessage를 export

④ view.js — render(state)를 옮기고 export

⑤ main.js — import 두 줄 · state · loadComments

⑥ main.js — 연결, index.html은 type="module"



모듈로 나눈 뒤 — 1번, 댓글 5개 (DAY 20과 같음)

실습 01 · 코드

밖에서 쓰는 것만 export

```
// api.js 15~26번 줄
// 2. 글 번호로 댓글 받기 – 주소 만드는 일은 통신 파일이 맡습니다
export function getComments(postId) {
  return getJSON(`${BASE_URL}/posts/${postId}/comments`);
}

// 3. 오류를 안내 문구로
export function toMessage(error) {
  if (error.name === "TimeoutError") return "응답이 너무 늦습니다";
  if (error.name === "TypeError") return "인터넷 연결을 확인해 주세요";
  if (error.status === 404) return "찾는 주소가 없습니다";
  return "잠시 후 다시 시도해 주세요";
}
```

```
// main.js 2~3번 줄
import { getComments, toMessage } from "./api.js";
import { render } from "./view.js";

// view.js 8번 줄
export function render(state) {
```

파일	맡은 일
api.js	통신 — 두 함수만 export
view.js	화면 — render(state)
main.js	연결 — 상태 · 요청 · 버튼

실습 01 · 확인

브라우저가 하나씩 받는 모듈 파일

Network 패널을 열고 새로고침하면, main.js가 가져오는 api.js · view.js가 따로 보입니다.
Initiator 칸에 누가 불렀는지 — main.js:2 · main.js:3 — 가 나옵니다.

Name	Status	Type	Initiator	Size	Time
practice01/	200	document	Other	0.9 kB	11 ms
style.css	200	stylesheet	practice01/:7	1.2 kB	3 ms
main.js	200	script	practice01/:22	1.3 kB	3 ms
api.js	200	script	main.js:2	1.2 kB	2 ms
view.js	200	script	main.js:3	1.4 kB	2 ms

Network 패널 — 요청 표 부분만 (캐시를 끄고 찍음)

확인할 것	보여야 하는 것
처음 · 1번 · 101번	처음 문구 · 댓글 5개 · 댓글이 없습니다 (DAY 20과 같음)
Network의 Initiator	api.js ← main.js:2, view.js ← main.js:3

실습 01

일부러 틀려 보기

완성한 파일을 한 곳씩 바꿔 저장하고, 결과를 본 뒤 되돌립니다.
네 가지 모두 화면은 비고, Console에 빨간 줄이 뜹니다.

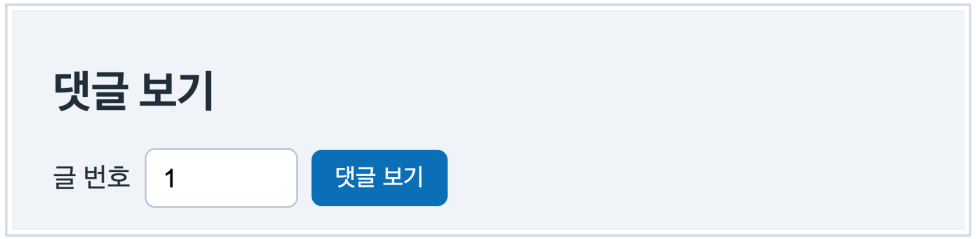
바꿀 곳	Console에 뜨는 것
① api.js 21번 줄 toMessage 앞 export 빼기	SyntaxError — does not provide an export named 'toMessage'
② index.html의 type="module" 빼기	SyntaxError — Cannot use import statement outside a module
③ main.js 2번 줄 "./api.js"를 "api.js"로	TypeError — Failed to resolve module specifier "api.js"
④ index.html을 file://로 직접 열기	CORS 오류 — Access to script ··· blocked by CORS policy

네 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 01 끝

‘내보내지 않았다’ — export 확인

```
// api.js 21번 줄 (일부러 틀린 곳 ①)
function toMessage(error) {

// main.js 2번 줄
import { getComments, toMessage } from "./api.js";
```



화면 — 처음 문구도 없음



Console (위쪽 도구 막대는 잘라 냄) — main.js 2번 줄 23번째 글자

‘없다’는 이름 — import 쪽 오타인지, export 쪽에서 빠졌는지 두 곳을 봅니다

여기까지 정리

1	export · import	내보내고 가져옴 — import { 이름 } from "./파일.js"
2	type="module"	index.html에는 시작점 하나, Live Server로 연다 (file:// 안 됨)
3	as · * as	이름 바꾸기 · 내보낸 것을 객체 하나로 한꺼번에
4	default	파일에 하나 — 중괄호 없이, 가져오는 쪽이 이름을 정함
5	모듈의 특징	자기 범위 · 자동 defer · 한 번만 실행

다음은 다른 사람이 만든 모듈을 받아 쓰는 법 — npm

02

NPM과 package.json

나눠 쓰는 코드 · npm · package.json · dependencies와 devDependencies · 버전

날짜 하나를 글자로 만드는 데도 필요한 코드

```
// today/main.js
const getToday = () => {
  const today = new Date();
  const year = today.getFullYear();
  const month = today.getMonth() + 1;
  const date = today.getDate();

  return `${year}년 ${month}월 ${date}일`;
};

console.log(getToday());
```

```
2026년 10월 9일
```

today/main.js — 날짜

```
// today_time/main.js
const getToday = () => {
  const today = new Date();
  const year = today.getFullYear();
  const month = today.getMonth() + 1;
  const date = today.getDate();
  const hours = today.getHours();
  const minutes = today.getMinutes();
  const seconds = today.getSeconds();

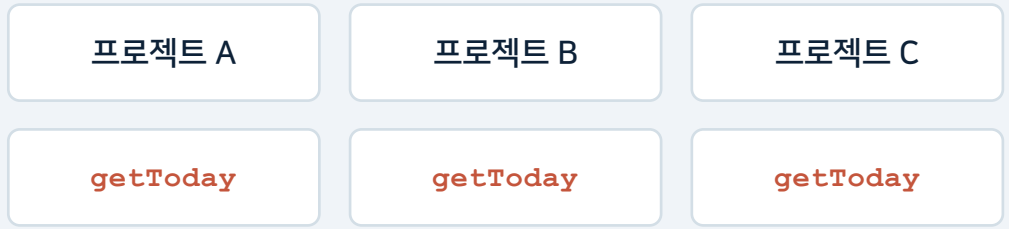
  return `${year}년 ${month}월 ${date}일 ${hours}시 ${minutes}분 ${seconds}초`;
};
  …
```

```
2026년 10월 9일 14시 36분 18초
```

today_time/main.js — 날짜와 시각

여러 번 쓸 코드는 모듈로 만들어 함께 쓰기

프로젝트마다 복사



고칠 때마다 세 곳을 따로 고침

저장소에서 받아 씀



고친 판이 올라오면 받기만 함

남이 만들어 올린 모듈 — 패키지 — 를 받아 쓰는 도구가 npm

오늘 실습 02는 날짜 코드를 내 모듈(date.js)로 나누고, 패키지를 받아 쓰는 것은 DAY 27부터.

02 · npm

패키지를 설치하고 관리하는 npm

WEBSITE

npmjs.com

패키지를 찾아보는 곳.
설명 · 버전 · 쓰는 법

CLI

npm 명령

터미널에서 패키지 설치 ·
업데이트 · 삭제를 관리

REGISTRY

레지스트리

패키지를 모아 둔 공개
데이터베이스 — 여기서 받음

npm은 세계에서 가장 큰 소프트웨어 레지스트리 (npm 문서)

npm은 Node.js를 설치하면 함께 들어옵니다 — 오늘은 개념만 보고, 직접 쓰는 것은 DAY 27.

프로젝트의 이름표 package.json

```
// package.json — 프로젝트마다 하나
{
  "name": "cafe-order",
  "version": "1.0.0",
  "dependencies": {
    "react": "^19.2.8",
    "dayjs": "^1.11.23"
  },
  "devDependencies": {
    "vite": "^8.3.0",
    "typescript": "^7.0.2"
  }
}
```

프로젝트 메타데이터

프로젝트의 이름과 판,
설명 같은 프로젝트 정보

의존성 관리

프로젝트가 동작하는 데 필요한
외부 패키지와 그 판을 적음
dependencies · devDependencies

dependencies와 devDependencies

PRODUCTION

dependencies

최종 제품에서 쓰는 패키지.
앱의 핵심 기능을 맡습니다

DEVELOPMENT

devDependencies

개발 · 테스트할 때만 쓰는 패키지.
테스트 · 빌드를 돕습니다

예 — dependencies : react(화면) · express(서버) / devDependencies : vite(빌드) · eslint(코드 품질) · typescript(타입)

프로젝트가 쓰는 패키지를 둘로 나눠 적습니다

package.json을 보면 프로젝트가 무엇을 쓰는지 한눈에 알 수 있습니다.

버전 번호 세 자리 — SemVer



MAJOR

1 → 2.0.0

호환이 깨지는 큰 변경.
전에 쓰던 코드가 안 될 수 있음

MINOR

11 → 1.12.0

호환되는 새 기능.
끝자리는 0으로

PATCH

23 → 1.11.24

호환되는 버그 수정.
가장 작은 변화

받아도 되는 판의 범위 ^ 와 ~

package.json에 적은 것	받을 수 있는 판	예
`"^1.11.23"`	1.11.23 이상, 2.0.0 미만	1.12.0은 됨 · 2.0.0은 안 됨
`"~1.11.23"`	1.11.23 이상, 1.12.0 미만	1.11.24는 됨 · 1.12.0은 안 됨
`"1.11.23"`	1.11.23 하나만	1.11.24도 안 됨

package-lock.json — 실제로 받은 정확한 판을 적어 두는 파일

범위는 package.json에, 실제로 받은 판은 package-lock.json에 적힙니다 — 다음에 받아도 같은 판이 됩니다.

여기까지 정리

1	npm	Node.js 패키지 관리 도구 — 패키지 설치 · 업데이트 · 삭제
2	package.json	프로젝트 정보와 쓰는 패키지를 적는 파일
3	dependencies	최종 제품에서 쓰는 패키지
4	devDependencies	개발 · 테스트할 때만 쓰는 패키지
5	버전	MAJOR.MINOR.PATCH — ^는 첫 자리, ~는 둘째 자리까지 고정

다음은 여러 파일을 묶어 주는 빌드와 번들러, 그리고 Vite

03

빌드와 Vite

빌드 · 번들러 · Vite

소스 코드를 제품으로 조립하는 빌드

소스 — 고치기 쉽게 나눈 파일

- main.js
- storage.js
- date.js
- style.css

받아 쓴 패키지

빌드 →

결과 — 브라우저에 보낼 파일

- index.html
- assets/index-BU0z5ubT.js
- assets/index-CMofWA48.css

빌드 — 개발자의 코드를 사용자에게 전달할 수 있는 제품으로 조립

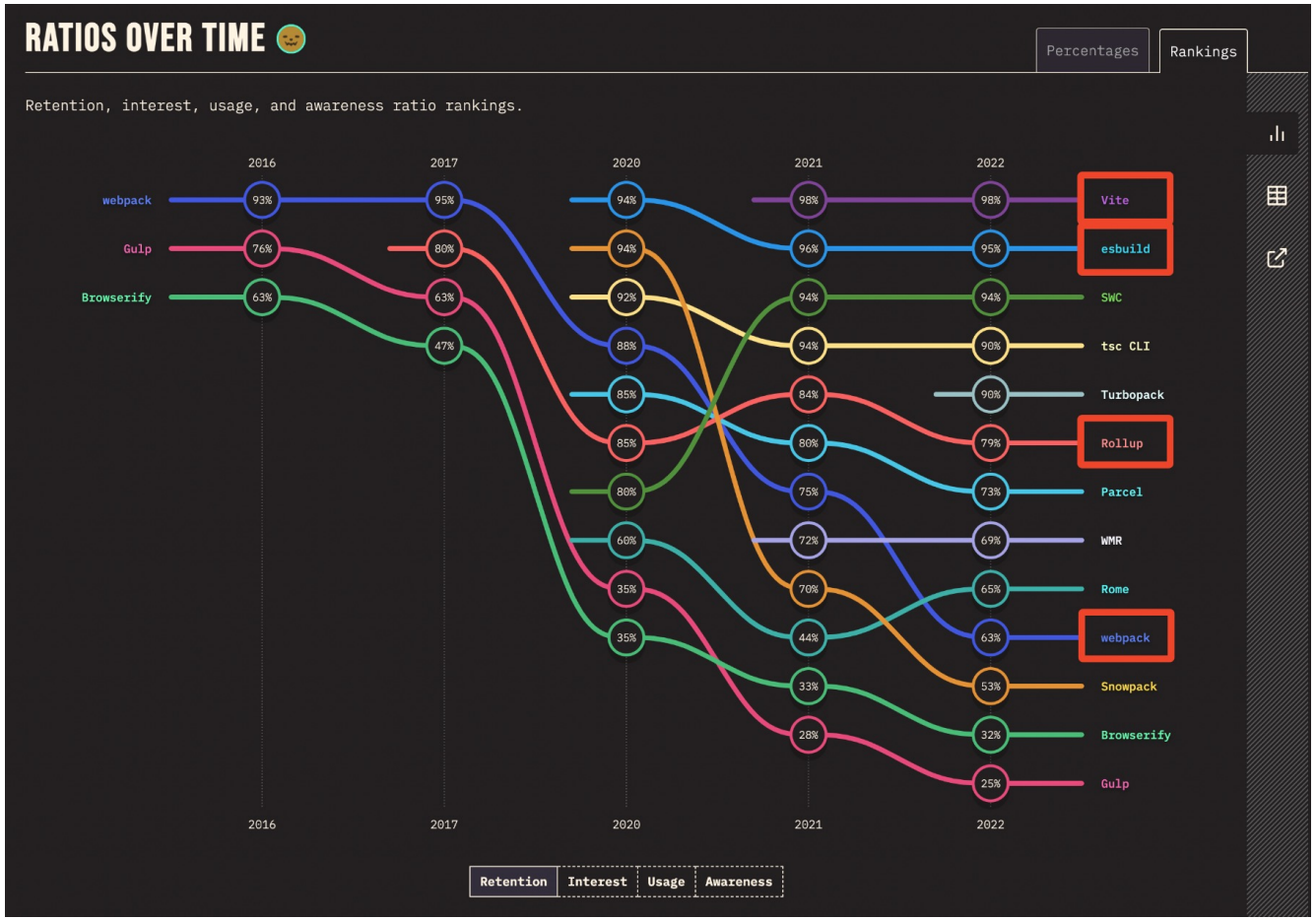
소스 코드를 배포 가능한 파일로 바꾸는 과정 — 번들링 · 최적화 · 압축 · 호환성 처리.

여러 파일을 묶어 주는 번들러

1	Webpack	모듈 번들링 · 자산(Assets) 관리 · 플러그인 생태계
2	Rollup	작은 코드 조각을 라이브러리나 앱으로 묶는 번들러
3	esbuild	Go로 작성해 아주 빠른 번들러
4	Vite	Vue.js를 만든 사람이 만든 프론트엔드 빌드 도구
5	Turbopack	Next.js의 기본 번들러

보통 번들러로 프론트엔드 개발 환경을 구성합니다

변화의 속도가 매우 빠른 번들러



빌드 도구를 다시 쓰겠다는 비율(Retention)의 순위 변화, 2016~2022 — State of JS

해마다 바뀌는 순위

webpack 93% → 63% (2016 → 2022)
Vite는 2021년에 나타나 98%

이름보다 하는 일

도구 이름은 바뀌어도
여러 파일을 묶어 제품으로
만드는 일은 같습니다

개발 서버와 빌드를 함께 맡는 Vite

DEV

개발 서버

브라우저의 ES 모듈로 바로 보여 주고,
고친 곳은 바로 화면에 (HMR)

BUILD

빌드

코드 분석 · 번들링 · 압축으로
배포할 파일을 만듦

프랑스어로 「빠르다」 — 읽기는 「비트」 (Vite 문서)

빠르고 간결한 개발 경험에 초점을 맞춘 빌드 도구

Vite 8은 Rolldown으로 묶습니다 — 직접 써 보는 것은 Node.js를 설치하는 DAY 27.

여기까지 정리

1	**빌드**	소스 코드를 배포 가능한 파일로 바꾸는 과정
2	**번들러**	여러 파일을 묶어 주는 도구 — Webpack · Rollup · esbuild 등
3	**Vite**	개발 서버(HMR)와 빌드를 함께 맡는 빌드 도구
4	**변화**	순위가 빠르게 바뀜 — 이름보다 하는 일을 기억
5	**직접 해 보기**	DAY 27에 Node.js를 설치하고 Vite로 React 프로젝트를 만듦

이제 DAY 20의 할 일 목록을 역할마다 모듈로 나눕니다

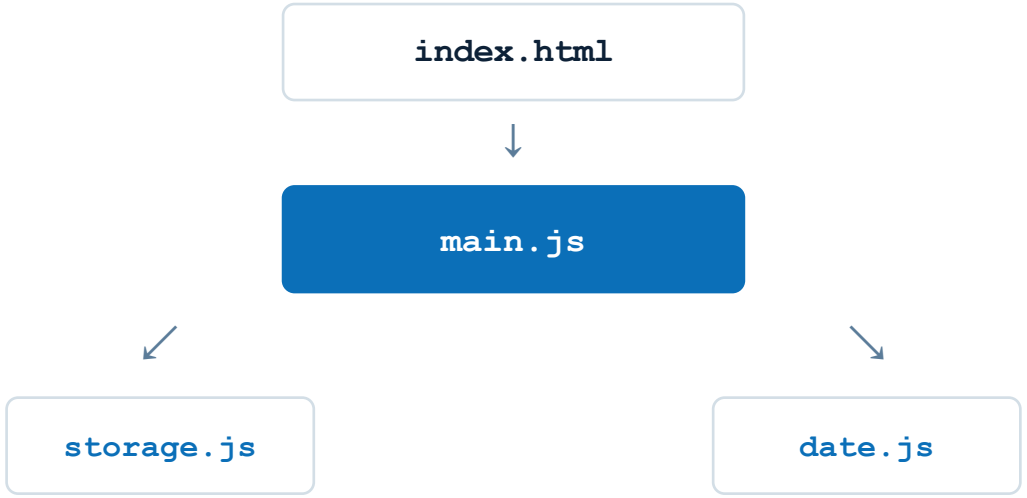
04

폴더 구조와 모듈 분리

역할마다 파일 하나 · 나누는 기준 · 실습 02

역할마다 파일 하나로 나눈 할 일 목록

```
practice02/
  index.html          ← 화면 뼈대 · 시작점
  style.css           ← 모양 (index.html의 link)
  main.js             ← 연결 (화면 · 이벤트)
  storage.js          ← 저장 (localStorage)
  date.js             ← 날짜 (Date)
  day20_todo.js       ← DAY 20 완성본 — 나눌 원본
```



import로 이어진 모양 — style.css는 index.html의 link로

파일을 나누는 기준

파일	맡은 일	export	가져다 쓰는 곳
`storage.js`	저장소에 넣고 꺼내기	loadTodos · saveTodos 외 3개	main.js
`date.js`	날짜를 글자로 (Date)	formatDate	main.js
`main.js`	화면 그리기 · 이벤트 연결	없음 — 시작점	index.html
`style.css`	모양	—	index.html (link)

한 파일에 한 가지 일, 다른 파일이 쓰는 것만 export

저장 방식을 바꾸려면 storage.js만, 날짜 모양을 바꾸려면 date.js만 고칩니다 — main.js는 그대로.

실습 02

할 일 목록을 역할마다 모듈로 나누기

DAY 20 실습 02의 todo.js를 저장 · 날짜 · 연결 세 파일로 나눕니다.
할 일마다 추가한 날짜를 붙이고, 파일끼리 export · import로 주고받습니다.

나누는 순서

① storage.js — 키 둘과 저장 함수 다섯, export
② date.js — formatDate(time)를 만들고 export
③ main.js — import 두 줄 · 불러오기 · 화면 그리기
④ main.js — 바꾸기 · 쓰던 글 · 추가하기
⑤ index.html — script를 type="module", main.js로
⑥ Live Server로 열어 날짜 · 새로고침 확인



할 일마다 추가한 날짜 — Date로 만든 「10월 9일」

실습 02 · 코드

저장과 날짜를 맡는 두 모듈

```
// storage.js 1~11번 줄
// 저장 — 할 일 목록은 localStorage, 쓰던 글은 sessionStorage
const TODOS_KEY = "day21-todos";
const DRAFT_KEY = "day21-draft";

export function loadTodos() {
  return JSON.parse(localStorage.getItem(TODOS_KEY)) || [];
}

export function saveTodos(todos) {
  localStorage.setItem(TODOS_KEY, JSON.stringify(todos));
}
// 13~23번 줄 — loadDraft · saveDraft · clearDraft도 export
```

TODOS_KEY · DRAFT_KEY는 안에서만 — export하지 않음

```
// date.js
// 날짜 — Date로 「10월 9일」 모양을 만듭니다
export function formatDate(time) {
  const date = new Date(time);
  return `${date.getMonth() + 1}월 ${date.getDate()}일`;
}
```

new Date(time)

time은 할 일의 id — 추가한 때의 Date.now()
getMonth()는 0부터 — 그래서 + 1 (28장)

실습 02 · 코드

가져와서 연결만 하는 main.js

```
// main.js 1~4번 줄 — 가져오기
// 실습 02 — 연결 : 할 일 배열을 바꾸고, 저장하고, 다시 그립니다

import { loadTodos, saveTodos, loadDraft, saveDraft, clearDraft } from "./storage.js";
import { formatDate } from "./date.js";
```

```
// 13번 줄 — 불러오기
let todos = loadTodos();

// 27~28번 줄 — render 안
    const dateEl = document.createElement("small");
    dateEl.textContent = formatDate(todo.id);
```

```
// 44~48번 줄 — 추가하기
function addTodo(text) {
  todos.push({ id: Date.now(), text: text, done: false });
  saveTodos(todos);
  render();
}
```

DAY 20 todo.js와 같은 흐름 — 저장은 saveTodos(todos), 날짜는 formatDate(todo.id)

여기까지 정리

1	**폴더 구조**	index.html · style.css 옆에 역할마다 JS 파일 하나
2	**나누는 기준**	한 파일에 한 가지 일 — 저장 · 날짜 · 연결
3	**export**	다른 파일이 쓰는 것만 — 안에서만 쓰는 것은 그대로
4	**시작점**	index.html은 main.js 하나만 — 나머지는 import로
5	**확인**	Live Server로 열고, 새로고침해도 목록이 남는지

파일을 나눠도 화면은 그대로 — 고칠 곳을 찾기가 쉬워졌습니다

점검

모듈에서 자주 나오는 세 가지

!

파일을 못 찾아요

import 경로의 오타와
./ · .js를 봅니다.
파일 이름과 한 글자씩.

!

export가 없다고 해요

내보내는 쪽에 export가
있는지, 중괄호 안 이름이
같은지 봅니다.

!

import를 못 쓴다고 해요

script에 type="module"이
있는지, Live Server로
열었는지 봅니다.

오류 문구에 파일 이름과 줄 번호가 있습니다 — 거기부터

화면만 보지 말고 Console을 함께 엽니다 — 빨간 줄의 파일 이름부터 봅니다.

WRAP UP

오늘 정리

개념

모듈 — 자기 범위를 가진 파일 하나

export · import — 이름으로 · 기본으로

npm — 패키지 설치 · 업데이트 · 삭제 관리

package.json — 프로젝트 정보 · 쓰는 패키지

버전 — MAJOR.MINOR.PATCH, ^ · ~

번들러 · Vite — 여러 파일을 묶는 도구

직접 쓸 수 있어야 하는 것

export · import로 파일 나누기

type="module"로 불러와 Live Server로 열기

역할마다 파일 — storage · date · main

dependencies · devDependencies 구분하기

빌드 · 번들러 · Vite가 하는 일 말하기

오류 문구에서 파일 이름과 줄 번호 찾기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

10장. NodeJS 시작

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

48장. 모듈

49장. Babel과 webpack을 이용한 ES6+/ES.NEXT 개발 환경 구축

NEXT

DAY 22

랜딩페이지 기획 및 구조 설계

랜딩페이지의 목적과 대상을 정하고, Hero · Feature · CTA · FAQ · Footer의 역할과 와이어프레임, 시맨틱 HTML 뼈대를 설계합니다.

수고하셨습니다.